

Project Report: DiskANN Vamana Index — Implementation, Optimization, and Analysis

DA24B004, DA24B033, DA24B049

1 Search Optimization: Proposals A–D

All experiments are conducted on the SIFT1M benchmark: one million 128-dimensional vectors, 10,000 queries, and top-100 ground truth, running on a native Windows MSVC C++ environment. The baseline index is built with $R = 32$, $L_{\text{build}} = 75$, $\alpha = 1.2$, $\gamma = 1.5$, completing in 93.3 seconds with an average out-degree of 33.1.

1.1 Flat Sorted Vector Candidate List

The standard priority queue implementation for the beam search candidate list uses a `std::set` (a Red-Black Tree), which allocates heap nodes per insertion and requires pointer-chasing for traversal. For candidate lists of size $L \leq 200$, this overhead is significant relative to the work done. We replace the `std::set` with a flat `std::vector` maintained in sorted order, using `memmove` for insertions. A separate `std::vector<bool>` tracks which nodes have been expanded via a zero-overhead boolean flag, eliminating redundant re-expansion checks. Contiguous memory layout ensures that candidate iteration stays within CPU cache lines rather than chasing pointers across the heap.

Table 1: Baseline exact float32 search with flat vector candidate list ($K = 10$, $R = 32$, $L_{\text{build}} = 75$).

L	Recall@10	Avg Dist Cmps	Avg Latency (μs)	P99 Latency (μs)
10	0.7734	641.7	766.0	2806.6
50	0.9662	1510.5	1679.1	5681.5
75	0.9819	1989.0	2079.4	5222.3
100	0.9882	2437.7	2568.0	8432.9
150	0.9938	3273.1	3362.2	8383.5
200	0.9961	4048.4	4753.2	23710.3

The flat vector forms the foundation on which the other proposals operate. By eliminating heap allocation latency and pointer indirection from the inner search loop, all subsequent optimizations (quantized distances, early abandonment, dynamic beam) benefit from a cleaner performance baseline where latency scales predictably with L .

1.2 Asymmetric Distance Computation via Scalar Quantization

Distance computation dominates the runtime of graph-based ANN search. The float32 representation (488 MB for SIFT1M) takes up memory bandwidth. To reduce this cost, we compress the dataset from float32 to uint8 using per-dimension min/scale scalar block quantization, reducing storage to 128 MB. During graph traversal, distances are computed asymmetrically: the query is kept in float32 while the data is represented in uint8. To prevent recall degradation from quantization noise, the full candidate pool is re-ranked using exact float32 distances before returning results.

Table 2: Exact Float32 vs. Quantized ADC on SIFT1M ($K = 10$, $R = 32$, flat vector candidate list).

L	Exact Recall	ADC Recall	Exact Avg Latency (μs)	ADC Avg Latency (μs)	Δ Latency
10	0.7734	0.7719	766.0	658.1	−14.1%
50	0.9662	0.9663	1679.1	1427.0	−15.0%
75	0.9819	0.9816	2079.4	2028.3	−2.5%
100	0.9882	0.9881	2568.0	2458.2	−4.3%
150	0.9938	0.9938	3362.2	3334.5	−0.8%
200	0.9961	0.9960	4753.2	3754.4	−21.0%

Recall is preserved across all L values. Latency improvements range from 0.8% to 21%, with the larger gains appearing at higher L values where the volume of distance comparisons is greatest and memory bandwidth becomes the bottleneck.

1.3 Early-Abandoning Distance Computation

For each distance comparison during graph traversal, the query is checked against a beam threshold. If the running sum of squared differences already exceeds the threshold partway through the 128 dimensions, the remaining computation is unneces-

sary. We integrate early-exit logic into both the exact and quantized distance functions, evaluating the accumulated distance against the current beam threshold at intervals of 16 dimensions. This interval aligns with AVX2/AVX-512 register widths.

Table 3: Exact Float32 without vs. with Early Abandonment (16-dimension intervals).

L	Recall@10	Dist Cmps	No-EA Latency (μs)	EA Latency (μs)	Improvement
10	0.7734	641.7	814.4	766.0	−6.0%
50	0.9662	1510.5	1741.7	1679.1	−3.6%
100	0.9882	2437.7	2746.8	2568.0	−6.5%
200	0.9961	4048.4	4845.8	4753.2	−1.9%

Latency savings range from 1.9% to 6.5%, achieved at essentially zero algorithmic cost.

1.4 Dynamic Beam Width

In standard beam search, the candidate list size L is fixed for the entire query. However, the optimal list size varies during traversal: at the start of a query, a larger list helps explore the graph broadly, while near convergence a smaller list suffices. We implement a dynamic heuristic that adapts the active list size L_{active} based on discovery saturation. When improvement stalls for a number of consecutive hops, L_{active} is expanded to escape local minima; when improvement is steady, L_{active} is contracted to reduce wasted computation.

Table 4: Standard vs. Dynamic beam width, exact float32 search.

L	Exact Recall	Dynamic Recall	Exact Latency (μs)	Dynamic Latency (μs)	Δ Latency
10	0.7734	0.7734	766.0	719.4	−6.1%
50	0.9662	0.9662	1679.1	1573.6	−6.3%
100	0.9882	0.9882	2568.0	2478.0	−3.5%
200	0.9961	0.9961	4753.2	3594.0	−24.4%

Table 5: Standard vs. Dynamic beam width, quantized ADC search.

L	Quant Recall	Dynamic Recall	Quant Latency (μs)	Dynamic Latency (μs)	Δ Latency
10	0.7719	0.7719	658.1	454.9	−30.9%
50	0.9663	0.9663	1427.0	1094.2	−23.3%
100	0.9881	0.9881	2458.2	1767.3	−28.1%
200	0.9960	0.9960	3754.4	2980.1	−20.6%

Recall is preserved exactly in both modes. Dynamic beam width delivers the largest latency gains of any single proposal: up to 24.4% reduction for exact search and up to 30.9% for quantized search.

Hyperparameter sensitivity. We perform a grid search over the shrink floor ratio F , expansion multiplier M , and hop limit H at $L = 200$ (baseline: 4753.2 μs , 0.9961 recall).

Table 6: Dynamic beam width hyperparameter grid at $L = 200$.

Configuration (F, M, H)	Recall@10	Dist Cmps	Avg Latency (μs)	Note
$F=0.5, M=2.0, H=10$ (standard)	0.9961	4048.4	3594.0	Optimal
$F=0.1, M=5.0, H=3$ (aggressive)	0.9961	4048.4	3760.1	Suboptimal

The moderate configuration ($F = 0.5$) outperforms aggressive expansion ($F = 0.1, M = 5.0$). Tight floor ratios force large beam expansions on stall events, increasing overhead without any recall benefit. Moderate parameters allow the beam to contract and expand smoothly without triggering expensive recovery.

2 Additional Optimizations Beyond Proposals A–D

This section describes further improvements to the Vamana index implemented on top of the baseline established by Proposals A–D. All experiments are conducted on SIFT1M ($K = 10$, top-100 ground truth) on Apple M-series hardware with OpenMP parallelism.

2.1 Algorithmic Correctness Fixes

Three aspects of the original codebase deviated from the algorithm described in the DiskANN paper, each of which independently degrades graph quality.

Medoid Initialization. The original code selects a random node as the search entry point during both build and query. The paper specifies that the entry point should be the *medoid* – the dataset point closest to the mean vector. The approximate medoid is computed in $O(n)$ by finding the dataset point nearest to the mean vector.

Random R -Regular Graph Pre-Initialization. We initialize every node with R randomly chosen neighbors before construction. This bootstraps connectivity so that early insertions have a meaningful graph to search through, producing better-quality edges throughout the build.

Full Visited Set to RobustPrune. During construction, after GreedySearch is run for a new node, the original code passes only the final top- L candidates to RobustPrune. The paper (Section 2.3) specifies that the *complete* set of all visited nodes V should be passed. The visited set is typically $2\text{--}4\times$ larger than the top- L list and contains nodes that were explored but ranked below the cutoff. These are precisely the candidates that become good long-range edges after RNG pruning. Discarding them forces the graph to use only nearby candidates, reducing navigability.

The combined effect of these three fixes is shown in Table 7. The most striking result is the P99 latency improvement: the fixes reduce worst-case latency by 60% at $L = 75$, from $2672.4\mu\text{s}$ to $1055.6\mu\text{s}$. This occurs because the medoid start node eliminates a class of pathological queries that previously required many extra hops to escape a poor starting region. Average latency improves by 25% at $L = 75$.

Table 7: Baseline vs. algorithmic fixes ($R = 32$, exact float32, single-pass build).

L	Baseline Recall	Fixed Recall	Baseline Avg (μs)	Fixed Avg (μs)	Baseline P99 (μs)	Fixed P99 (μs)
10	0.7820	0.7993	174.4	187.1	862.4	1693.1
50	0.9661	0.9675	450.7	347.0	1978.4	727.9
75	0.9818	0.9821	627.8	474.4	2672.4	1055.6
100	0.9883	0.9881	760.0	596.6	2802.1	1357.1
200	0.9960	0.9958	1427.4	1026.7	5076.8	2155.0

2.2 Higher Graph Degree ($R = 64$)

The graph degree R controls how many neighbors each node retains after pruning. With $R = 32$, each node has at most 32 outgoing edges. Doubling to $R = 64$ gives RobustPrune a larger and more diverse set of edges to retain per node, covering more directions in the embedding space. This directly reduces the number of hops a query needs to reach its true nearest neighbors, since the probability that a given node has an edge pointing toward the query region increases with degree.

The build uses $L_{\text{build}} = 100$ (rather than 75) to give GreedySearch a larger candidate pool during construction, ensuring RobustPrune has enough options to populate 64 high-quality edges per node. Build time increases from 110 seconds ($R = 32$) to 338 seconds ($R = 64$) — a one-time cost. The $R=64$ index is searched with Quantized ADC (Proposal A) to keep per-query latency competitive.

Table 8: $R = 32$ (Quantized ADC) vs. $R = 64$ (Quantized ADC).

L	$R=32$ Recall	$R=64$ Recall	Δ Recall	$R=32$ Latency (μs)	$R=64$ Latency (μs)
10	0.8041	0.8914	+0.087	140.8	231.0
20	0.9036	0.9622	+0.059	161.1	268.1
30	0.9405	0.9824	+0.042	210.0	359.6
50	0.9708	0.9936	+0.023	312.8	521.7
75	0.9848	0.9974	+0.013	420.7	707.4
100	0.9906	0.9984	+0.008	535.0	916.0
200	0.9968	0.9992	+0.002	934.0	1613.5

Recall improves by 0.002–0.087 across all L values. Average latency at the same L is 64–73% higher, since the denser graph requires more distance computations per expansion.

2.3 Best Overall Configuration

The best configuration combines all algorithmic fixes with $R = 64$ and Quantized ADC search.

```
# Build
./build_index --data sift_base.fbin --output sift_index_best.bin \
  --R 64 --L 100 --alpha 1.2 --gamma 1.5 --single_pass

# Search
./search_index --index sift_index_best.bin --data sift_base.fbin \
  --queries sift_query.fbin --gt sift_gt.ibin \
  --K 10 --L 10,20,30,50,75,100,150,200 --quantized
```

Table 9: Original baseline vs. best configuration on SIFT1M.

L	Baseline Recall	Best Recall	Baseline Avg (μs)	Best Avg (μs)	Baseline P99 (μs)	Best P99 (μs)
10	0.7820	0.8914	174.4	231.0	862.4	1519.2
20	0.8900	0.9622	252.6	268.1	1279.8	569.2
30	0.9334	0.9824	300.2	359.6	852.8	801.7
50	0.9661	0.9936	450.7	521.7	1978.4	1102.1
75	0.9818	0.9974	627.8	707.4	2672.4	1426.1
100	0.9883	0.9984	760.0	916.0	2802.1	1884.9
150	0.9936	0.9989	1084.3	1271.4	4183.8	2455.8
200	0.9960	0.9992	1427.4	1613.5	5076.8	3241.9

Recall improves at every L value, with gains of up to +0.109 at $L = 10$. P99 latency improves by 44–74% across mid-range L values despite the denser graph, because the medoid initialization eliminates the hard queries that previously drove worst-case latency. The most meaningful comparison is at equivalent recall, shown in Table 10.

Table 10: Equivalent-recall comparison: baseline vs. best configuration.

Recall	Configuration	L	Avg Latency (μs)	P99 Latency (μs)	Avg Dist Cmps
≈ 0.982	Baseline	75	627.8	2672.4	1995.4
	Best	30	359.6	801.7	1911.7
0.9936	Baseline	150	1084.3	4183.8	3279.8
	Best	50	521.7	1102.1	2731.3
≈ 0.997	Baseline	200	1427.4	5076.8	4054.0
	Best	75	707.4	1426.1	3660.9

At all three recall levels the best configuration achieves 43–52% lower average latency and 70–74% lower P99 latency using significantly fewer distance computations.

2.4 Improvements That Did Not Help

Several additional techniques were tested and found to be either neutral or harmful on SIFT1M, as summarized in Table 11.

Table 11: Tested improvements with negative or neutral outcomes.

Improvement	Outcome
Multiple entry points ($k = 8$)	No benefit. SIFT1M lacks strong cluster structure, so all entry points converge near the medoid and provide no routing advantage over a single entry point.
Strict γ removal ($\gamma = 1.0$)	–0.02 recall at $L = 75$ with only –15% latency. The recall drop outweighs the gain.
PCA traversal (32-dim)	Recall collapses to 0.50 at $L = 75$. SIFT descriptors have high intrinsic dimensionality; 32 dimensions are insufficient to preserve neighbor ranking.
PCA traversal (64-dim)	Recall 0.84 at $L = 75$, still 0.14 below exact float32. Not competitive.
Quantization-aware graph refinement	–0.006 recall. Re-pruning with quantized distances removes edges without adding new ones, leaving the graph sparser and less navigable.
Two-pass build ($R = 64$)	Bus error on Mac due to memory pressure during Pass 2 on the dense $R = 64$ graph.

2.5 Final Configuration and Results

The final configuration combines all improvements from both Proposals A–D and our additional optimizations:

- **Graph degree** $R = 64$, $L_{\text{build}} = 100$, $\alpha = 1.2$, $\gamma = 1.5$, single-pass build
- **Medoid initialization**, random R -regular graph pre-initialization, full visited set V to RobustPrune
- **Quantized ADC search** — 8-bit scalar quantization during traversal, exact re-ranking of top- L candidates
- **Early-abandoning distance** — always active by default
- **Dynamic beam width** — floor ratio $F = 0.5$, expansion multiplier $M = 2.0$, hop limit $H = 10$

- **Flat sorted vector candidate list** — always active by default

```
./build_index --data sift_base.fbin --output sift_index_final.bin \
--R 64 --L 100 --alpha 1.2 --gamma 1.5 --single_pass

./search_index --index sift_index_final.bin --data sift_base.fbin \
--queries sift_query.fbin --gt sift_gt.ibin \
--K 10 --L 10,20,30,50,75,100,150,200 \
--quantized --dynamic \
--dyn-floor-ratio 0.5 --dyn-exp-mult 2.0 --dyn-hops 10
```

2.5.1 Equivalent-Recall Comparison (Primary Result)

The most meaningful comparison between the Proposals A–D baseline and the final configuration is at equivalent recall, since the denser $R = 64$ graph requires fewer hops per query. Table 12 shows that at every recall level tested, the final configuration achieves substantially lower average and P99 latency using a lower L value.

Table 12: Equivalent-recall comparison: Proposals A–D baseline vs. final configuration.

Recall Δ P99	Configuration	L	Avg Latency (μ s)	P99 Latency (μ s)	Δ Avg
≈ 0.982 –40%	Baseline (A–D)	75	445.6	1027.2	–22%
	Final	30	349.0	617.9	
0.9936 –78%	Baseline (A–D)	150	944.3	4801.6	–46%
	Final	50	513.2	1049.6	
≈ 0.997 $\approx 0\%$	Baseline (A–D)	200	954.4	2024.4	–21%
	Final	75	755.9	2144.8	

The strongest result is at 0.9936 recall: the final configuration achieves this with $L = 50$ in 513.2μ s, compared to $L = 150$ at 944.3μ s for the baseline — a **46% reduction in average latency** and **78% reduction in P99 latency** at identical recall.

3 Use of AI-Based Coding Tools

3.1 Tools Used

We used **Claude** as an AI coding assistant throughout the project, primarily for implementation guidance, code generation, and debugging support.

3.2 Tasks Where AI Was Effective

Task	AI Contribution	Effectiveness
Boilerplate generation	CLI parsing, I/O utilities, CMakeLists	Very effective
Algorithm explanation	α -RNG pruning, beam search, ADSampling	Good for intuition
Code scaffolding	Struct definitions, function signatures	Good starting points
Debugging	Identifying locking and memory issues in parallel build	Correct diagnosis
Early-abandon design	Chunked loop structure with SIMD-friendly boundaries	Hardware-aware
Research survey	Identifying post-DiskANN improvements (FINGER, ADSampling, RaBitQ)	Comprehensive

Table 13: Tasks where AI tools were effective.

3.3 Tasks Where AI Failed or Required Correction

Failure Mode	Human Correction Required
FINGER implementation produced $10\times$ slower search with 0.33 recall	Reverted entirely; root causes were incorrect angle formula, random rather than data-driven basis vectors, and per-query allocations of npts-sized vectors
PCA traversal presented as promising	Benchmarks showed recall collapse to 0.50 at $L = 75$; AI had underestimated SIFT1M’s intrinsic dimensionality
Cache-friendly graph reordering remapped node IDs in memory but not on disk	Caused catastrophic recall loss (0.0004); required identifying and disabling the feature
Two-pass build with $R = 64$ suggested thread reduction as fix for bus error	Memory pressure persisted at any thread count; single-pass build was the correct resolution
Quantization-aware refinement predicted +0.5–1% recall gain	Benchmarks showed -0.006 recall; re-pruning without adding candidates made the graph sparser

Table 14: AI failures and required human corrections.

3.4 Key Limitation

The most important lesson: AI can generate correct-looking code and plausible analysis, but it cannot run code or measure performance. In several cases the AI produced implementations that were algorithmically reasonable but experimentally harmful. Every number in this report comes from actual execution on real hardware. Experimental validation was always required before accepting any AI-suggested change.

4 Critical Reflection and Takeaways

4.1 Where Human Judgment Was Essential

- Identifying the full visited set bug.** The AI initially passed only the top- L candidates to **RobustPrune** rather than the complete visited set V . Carefully reading Section 2.3 of the paper revealed this discrepancy. Fixing it produced the largest single quality improvement in the project — a 60% reduction in P99 latency at $L = 75$.
- Interpreting negative results.** Multiple techniques that appeared promising in theory — FINGER, PCA traversal, multi-entry-point search, quantization-aware refinement — all failed on SIFT1M. Recognizing *why* they failed (high intrinsic dimensionality, lack of cluster structure, graph sparsification) required understanding the dataset, not just running the code.
- Debugging platform-specific crashes.** The two-pass build with $R = 64$ crashed with a bus error on Mac due to memory pressure on a dense graph. The AI suggested thread-count reductions as a fix; the correct resolution was to use single-pass build, which required understanding the memory footprint of the graph construction process.
- Deciding what to compare.** The AI initially compared configurations at the same L value, which made the $R = 64$ results look worse due to higher per- L latency. The insight that equivalent-recall comparison is the correct framing — and that a higher-recall graph at lower L beats a lower-recall graph at higher L — required domain understanding, not code.

4.2 How We Guided the AI

- Paper-grounded specification:** Rather than asking the AI to “improve the index”, we provided specific section references from the DiskANN paper, e.g., “implement the medoid as described in Section 2.3, paragraph 2, not a random start node”.
- Iterative validation:** Every AI-generated change followed a cycle of: implement $\rightarrow$ compile $\rightarrow$ benchmark $\rightarrow$ compare against baseline $\rightarrow$ accept or revert.
- Insisting on experimental evidence:** When the AI predicted an improvement, we always ran the benchmark before accepting the claim. Several confident predictions were contradicted by results.

4.3 Lessons Learned

- Graph construction quality dominates search-time optimization.** Proposals A–D reduced latency by approximately 30% at the same recall. Build-time improvements (medoid, $R = 64$, full visited set) achieved 46% lower latency and 78% lower P99 at equivalent recall. Investing in graph quality is the higher-leverage strategy.
- Negative results are scientifically valuable.** PCA traversal, multi-entry-point search, FINGER, and quantization-aware refinement all failed on SIFT1M, but each failure revealed a structural property of the dataset or an inherent limitation of the technique. These results constrain the design space for future work.

3. **Mean latency and P99 latency measure different things.** Mean latency reflects average-case behavior; P99 reflects the hardest queries — those whose nearest neighbors lie in poorly connected graph regions. The medoid initialization and denser graph improved P99 by up to 78%, far more than the mean improvement, because they specifically address the structural causes of hard queries.
4. **Platform portability matters.** Behaviors that are stable on one platform — memory allocation, OpenMP thread-local state, hardware prefetching — can fail or behave differently on another. Testing on the target hardware (Apple M-series) rather than assuming x86 behavior was necessary throughout.

5 Conclusion

Starting from the baseline Vamana index ($R = 32$, exact float32 search), we first implemented four search-time optimizations (Proposals A–D): quantized ADC distance computation, early-abandoning distance evaluation, dynamic beam width, and a flat sorted vector candidate list. Building on this, we implemented three build-time correctness fixes from the DiskANN paper — medoid initialization, random R -regular graph pre-initialization, and passing the full visited set to RobustPrune — and increased the graph degree to $R = 64$. The combined effect of all improvements, measured on SIFT1M ($K = 10$), is summarized below.

Table 15: Summary of results at equivalent recall levels.

Recall	Configuration	L	Avg Latency (μ s)	P99 Latency (μ s)
≈ 0.982	Proposals A–D baseline	75	445.6	1027.2
	Final (all improvements)	30	349.0	617.9
0.9936	Proposals A–D baseline	150	944.3	4801.6
	Final (all improvements)	50	513.2	1049.6

At 0.9936 recall, the final configuration requires $L = 50$ compared to $L = 150$ for the baseline, achieving **46% lower average latency and 78% lower P99 latency**. Recall improves at every L value, with gains of up to +0.113 at $L = 10$.

We also tested several additional techniques — FINGER approximate distance, PCA-based traversal, quantization-aware graph refinement, and multi-entry-point search — all of which produced neutral or harmful results on SIFT1M and are documented as negative results in Section 2.4. These failures highlight that SIFT1M’s high intrinsic dimensionality and lack of strong cluster structure limit the effectiveness of techniques that assume low-dimensional or clustered data.

The principal finding is that build-time graph quality improvements deliver larger and more robust gains than search-time approximations on this benchmark. A denser, better-initialized graph allows the search to use a smaller beam width to achieve the same recall, reducing both mean and tail latency simultaneously.

6 Repository

<https://github.com/vrishabav/GraphANN-Improved/tree/main>